

Real-Time Task Board / Todo

Overview

We need a lightweight task management board where users can create, update, and move tasks across different stages. The focus is on real-time collaboration—changes made on one device should reflect immediately on other devices.

This is not meant to be a full project management tool. The goal is to build a fast, responsive board with basic collaboration and conflict handling.

Scope

In Scope

- Task creation, editing, deletion
 - Moving tasks between stages
 - Real-time sync across multiple clients
 - Handling update conflicts (multi-device edits)
 - Basic search and filtering
 - Simple User login/signup
-

Task Lifecycle

Each task will move through the following stages:

- **Todo**
- **In Progress**
- **Done**

A task should always belong to one of these states.

Functional Requirements

1. Task Management

Users should be able to:

- Create a new task with:
 - Title (required)
 - Description (optional)
 - Edit an existing task
 - Delete a task
-

2. Task Movement

- Tasks should be draggable between columns:
 - Todo → In Progress → Done
 - Movement should feel instant on the UI
 - Backend should persist the updated state
-

3. Real-Time Updates

- Any change (create/update/delete/move) should reflect across all connected clients in near real-time
- No manual refresh should be required

Example:

- User A updates a task → User B sees it immediately
-

4. Conflict Handling (Multi-Device)

This is important.

Scenario:

- Same task is edited from two devices at the same time

Approach:

- Each task will store an **updatedAt** timestamp
- The system will follow **last update wins**

Rule:

- If an incoming update has a newer timestamp than the current one, it should overwrite the existing data
- Otherwise, ignore it

This should be handled at the backend level to maintain consistency

5. Search & Filter

- Users should be able to:
 - Search tasks by title
 - Filter tasks by status (Todo / In Progress / Done)

Search input should be debounced to avoid excessive API calls

Non-Functional Requirements

Performance

- UI should remain responsive even with ~100+ tasks
 - Avoid unnecessary re-renders (especially during drag & drop)
 - Updates should feel instant
-

Reliability

- System should not break if multiple updates happen quickly
 - Real-time sync should be stable even with multiple clients
-

Consistency

- All clients should eventually reflect the same state
 - Conflict handling should prevent data corruption
-

Data Model (High-Level)

Task:

- id
 - title
 - description
 - status (todo | in-progress | done)
 - createdAt
 - updatedAt
-

API Requirements

Basic REST endpoints:

- GET /tasks → fetch all tasks
- POST /tasks → create task
- PUT /tasks/:id → update task
- DELETE /tasks/:id → delete task

Real-time events should mirror these actions

Real-Time Communication

Use a socket-based approach for:

- Task creation
- Task updates
- Task deletion
- Task movement

Clients should subscribe to updates and apply them immediately

UI Expectations

Layout

- 3-column board (Todo, In Progress, Done)
- Clear separation between columns

Task Card

- Title (visible)
 - Short description (optional preview)
 - Last updated time
-

Interaction Details

- Drag & drop should be smooth
 - Editing should not feel disruptive (modal or inline is fine)
 - Loading and empty states should be handled
-

Edge Cases to Consider

- Rapid updates from multiple devices
 - Task deleted while being edited elsewhere
 - Empty task list
 - Slow network / delayed socket events
-

Acceptance Criteria

- Tasks can be created, edited, deleted without issues
 - Drag & drop works reliably
 - Changes reflect in real-time across devices
 - Conflict resolution works using timestamps
 - Search and filtering behave correctly
 - UI remains responsive and usable
-

Notes

- Keep implementation simple but clean
 - Focus more on correctness and performance than UI polish
-